

RIDS: Towards Advanced IDS via RNN Model and Programmable Switches Co-Designed Approaches

Ziming Zhao¹, Zhaoxuan Li², Zhuoxue Song¹, Fan Zhang^{1,*}, Binbin Chen³

¹Zhejiang University, China

²SKLOIS, Institute of Information Engineering, Chinese Academy of Sciences, China

³Singapore University of Technology and Design, Singapore

{zhaoziming, songzhuoxue, fanzhang}@zju.edu.cn, lizhaoxuan@iie.ac.cn, binbin_chen@sutd.edu.sg

Abstract—Existing Deep Learning (DL)-based network Intrusion Detection System (IDS) is able to characterize sequence semantics of traffic and discover malicious behaviors. Yet DL models are often nonlinear and highly non-convex functions that are difficult for in-network deployment. In this paper, we present RIDS, a hardware-friendly Recurrent Neural Network (RNN) model that is co-designed with programmable switches. As its core, RIDS is powered by two tightly-coupled components: (i) rLearner, the RNN learning module with in-network deployability as the first-class requirement; and (ii) rEnforcer, the concrete pipeline design to realize rLearner-generated models inside the network dataplane. We implement a prototype of RIDS and evaluate it on our physical testbed. The experiments show that RIDS could satisfy both detection performance and high-speed bandwidth adaptation simultaneously, when none of the other existing approaches could do so. Inspiringly, RIDS realizes remarkable intrusion/malware detection effect (*e.g.*, ~99% F1 score) and model deployment (*e.g.*, 100 Gbps per port), while only imposing nanoseconds of latency.

Index Terms—Intrusion/malware detection system, hardware-friendly deep learning, programmable switches

I. INTRODUCTION

Intrusion Detection Systems (IDS) is one of the critical security mechanisms for detecting on-going malicious activities in networked systems. Over the past decades, the proposed approaches in the IDS landscape gradually evolve from policy-based [49] to deep learning-based (DL-based) detection [2], [44]. The former aims to describe the special pattern for each type of attack, so it depends on the knowledge completeness of attack patterns. The latter has superior generalization capability [31], especially being able to extract the temporal semantics of traffic such as Recurrent Neural Network (RNN) [27]. However, the existing DL models do not perform line-speed detection in the high bandwidth, *e.g.*, the recent approach reports a 4.7-millisecond inference time with GPU acceleration, which can barely saturate a 1 Mbps link if performing model inference for each instance [9].

* Fan Zhang is the corresponding author. This work was supported in part by National Natural Science Foundation of China (62227805, 62072398, 62172405), by SUTD-ZJU IDEA Grant for visiting professors (SUTD-ZJUV201901), by the Natural Science Foundation of Jiangsu Province (BK20220075), by the Fok Ying-Tung Education Foundation for Young Teachers in the Higher Education Institutions of China (No.20193218210004), by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Open Foundation of Henan Key Laboratory of Cyberspace Situation Awareness (HN2022001).

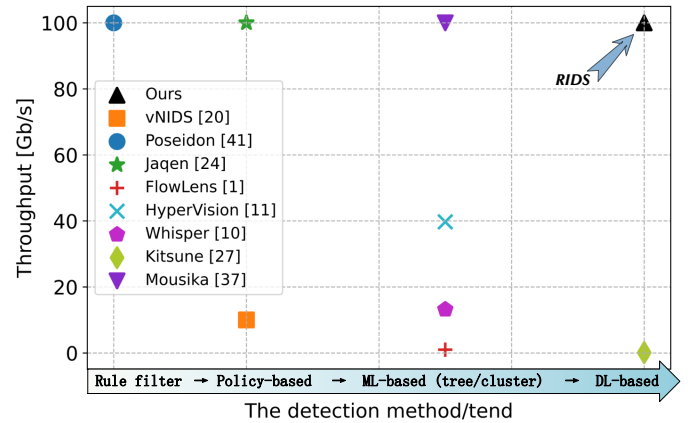


Fig. 1: The detection method/tend (stated in § II-A) with throughput diagram, comparing RIDS with eight representative IDS baselines. It is clear that RIDS realizes the powerful DNN model capability (*e.g.*, RNN) and high-speed bandwidth (*i.e.*, 100 Gbps per port, and the aggregated throughput can reach 3.2Tbps on the typical 32-port programmable switches).

We outline eight representative IDS and draw the detection method (development trend) with throughput in Figure 1. Existing detection systems either perform limited model capability or cannot adapt to high-speed bandwidth. For example, Mousika [44] deploys the decision tree in the data plane to guarantee line-speed processing, but cannot maintain flow-level information. Regarding Deep Neural Network (DNN) methods, Kitsune [31] puts effort into making AutoEncoder-based detection run efficiently, while it only supports ~112 Mb/s throughput, but still has a big gap with modern 100 Gbps bandwidth. Considering the importance of real-time detection in safety-critical scenarios (*e.g.*, facilitate timely countermeasures, as stated in § II), we tend to implement both complex model capability and high bandwidth adaptation requirements, *i.e.*, achieving the black triangle in the upper right part of Figure 1.

With developments of Application Specific Integrated Circuit (ASIC), recently emerging programmable switches allow user-customized packet processing logic in a protocol-independent manner. It motivates us to design hardware-friendly DNN inference to embrace this progress. Towards this end, we recognize two major challenges in realizing on-chip intelligent

traffic analysis. (i) Offloading feature extraction and flow state maintenance to the switches. Typical feature extraction, without strict timing requirements, tends to be comprehensive and requires complex calculations. Thus, the first step is to realize line-speed feature extraction and flow state maintenance. (ii) Achieving tailor-made model inference to cope with hardware resource constraints. A more crucial problem is to enable on-chip points to process packets based on the inference results. DNN models are either nonlinear and highly non-convex functions or involve lots of multiplication and activation operations, whereas programmable switches usually have limited computation power, *e.g.*, do not support multiplication and floating number computations. So designing the hardware-friendly DL models inside the switches is non-trivial.

In this paper, we present RIDS (RNN-based IDS, for short), a co-designed system achieving DL-driven intrusion detections at network line-speed by leveraging the emerging hardware primitive. RIDS is powered with two tightly coupled components named as rLearner and rEnforcer. rLearner is an RNN-based learning module, focusing on implementing the hardware-friendly model inference process which only depends on bitwise operations or integer additions/subtractions. To tackle this task, rLearner internally uses a number of techniques (*e.g.*, ternary weights, binary activation functions, and decoupled stateful tables) to architect its models.

rEnforcer is the specially designed pipeline enforcing rLearner-generated models inside the switches. rEnforcer leverages the hardware primitives on programmable switches, which can be reconfigured in the field without additional hardware upgrades. We architect rEnforcer as an extensible scheduling framework centering around a series of modules (such as feature collector, matrix processor, and table storer) to host the RNN models delivered by rLearner. These modules, supported by switch primitives (*e.g.*, register and control flow), collectively enable rEnforcer to store per-flow state and classify attack traffic based on model inference results.

Contributions. Our contributions are summarized as follows:

- We design the hardware-friendly inference processing for the RNN model, which only requires bitwise operations or integer additions/subtractions, including ternary weights, binary activation functions, and decoupled stateful tables.
- We leverage the emerging programmable switches primitives and architect the tailor-made RNN model as a series of dataplane modules to realize the line-speed detection.
- We implement a prototype of RIDS on our physical testbed, and conduct extensive evaluations to demonstrate the advantages of RIDS. The experiments show that RIDS realizes >99% F1 score for intrusion/malware detections, while only imposing negligible overhead, *e.g.*, nanoseconds of latency on a 100 Gbps bandwidth.

II. BACKGROUND AND MOTIVATION

A. Existing Intrusion Detection Systems

Existing intrusion detection systems can be roughly categorized into policy-based, machine learning-based (ML-based),

and deep learning-based (DL-based) approaches [53]. (i) Policy-based methods aim to profile the special pattern and set up primitives accordingly for each attack type. Li *et al.* [25] proposed vNIDS that shares detection state to achieve effective intrusion detection. Poseidon [49] and Jaqen [28] map the defense primitives to run on programmable switches for DDoS mitigation [51]. However, the effectiveness of these approaches often depends on prior knowledge about attack signatures. As a result, variations and combinations of known attacks could render policy-based methods unreliable to some extent.

(ii) Unlike policy-based approaches, ML-based IDS extracts a series of features and uses machine-learning algorithms to fit data, such as Markov chains [37]. Realizing the importance of deployment, Whisper [14] and HyperVision [15] adopt Intel's Data Plane Development Kit (DPDK) [33] to deploy their model into the data plane. Nonetheless, given limitations at the software level, they can support about 10~50 Gbps throughput and introduce a latency of a few seconds.

For combining ML models and programmable switches, the early schemes [4], [7], [23], [43] do not provide switch hardware implementation and use the BMv2 emulator [8] instead, which results in the inability to adapt to high bandwidth. FlowLens [2] realizes feature extraction on the data plane, but performs decision tree inference in the control plane. The following work [1], [44], [47], [54], [55] considers switch constraints and implements tree-based model hardware deployment. It is natural since the tree-based model is essentially a series of *if-else* branches. Some earlier attempts of neural networks are N2Net [38] and BaNaNa Split [35]. Also, Mousika [44] proposes the Binary Decision Tree distilled from RNN models to deploy on the programmable switches for packet-level classification. However, directly deploying DNN models is still difficult since they are either nonlinear or highly non-convex.

(iii) DL-based approaches embrace the advances in deep neural networks to identify attack traffic, *e.g.*, Recurrent Neural Network (RNN) [27], [40] and Convolutional Neural Network (CNN) [36], [50]. ET-BERT [26] characterizes the traffic contextualized datagram with pre-training transformers. The above method does not consider the deployability of the model. Kitsune [31] dedicates to developing a low-complexity model to reduce overhead and proposes to use AutoEncoder to detect abnormal behavior, but only can support ~112 Mbps. A common limitation shared by these approaches is that they are *not designed with line-speed deployment as the top priority, and essentially offline analyses rather than a real-time defense.*

However, real-time intrusion detection is crucial in security-critical scenarios, and practitioners can timely enable corresponding measures according to the detection results, such as disconnecting malicious IPs. Overall, real-time requirements largely limit the power of DL models in practice, so we intend to advance the co-designed research for deep learning and emerging programmable dataplane primitives.

B. Opportunities and Constraints of Programmable Switches

Dataplane programmability is the recent trend in networking innovations. In contrast to OpenFlow-based software-defined

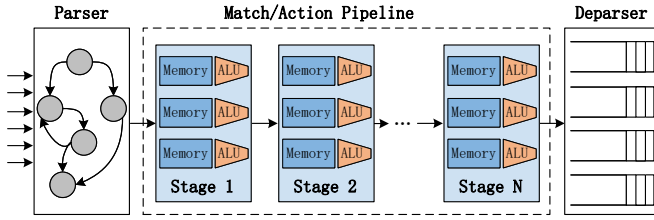


Fig. 2: Protocol Independent Switch Architecture (PISA).

networking, P4-programmable networks provide new features that can be reconfigured directly in hardware [3]. (i) *Opportunities*. As Figure 2 shows, packets first go through a programmable parser, which supports user-defined protocols. Meanwhile, packet headers are then steamed through a number of hardware stages, each of which contains stateful registers, Arithmetic Logic Units (ALUs), and match/action tables. Such Protocol Independent Switch Architecture (PISA) can perform computations at line speed, facilitating the implementation of terabit packet processing. Compared with software-based solutions [11], [21], programmable switches achieve higher throughput and lower latency. Besides, using domain-specific languages (e.g., P4 [3]), operators can customize the dataplane logic. Specifically, it allows us to programmatically maintain per-flow information and perform computations in ALUs.

Such new hardware primitives have attracted great attention. The community has proposed to use programmable switches in various areas, such as load balancing [18], [30], in-network measurement [5], [16], [32], [39], [48], application-level acceleration [20], [21] and security [22], [45], [46], [49]. Following this trend, RIDS explores the design of achieving line-speed DL-driven intrusion detection based on programmable switches.

(ii) *Constraints*. However, the new primitives of programmable switches have limitations. Switch ASIC contains a very small amount of memory and only a fraction of the total available SRAM (Static Random-Access Memory) can be used to allocate register arrays. There also are other constraints due to the special memory architecture of switching ASICs. For example, we cannot access the registers of one stage from other stages. Moreover, since SRAM is uniformly distributed in different pipeline stages, it is not straightforward to access all available registers. To ensure line-speed packet forwarding, the switches restrict the total number and type of operations allowed within each stage. Specifically, multiplications, divisions or floating-point operations, and variable-length loops are not supported. And each action on tables can only perform a restricted set of simple operations, such as additions, bitwise shifts, and memory accesses [2].

Thus, although promising, programmable switches are not panaceas, *i.e.*, it is simply infeasible to execute arbitrary DL models on switches. RIDS is therefore designed to fulfill on-switch DL inference despite these constraints.

III. ARCHITECTURE AND OVERVIEW

In Figure 3, we depict a high-level architecture of RIDS, including the key building blocks of each component. Among them, rLearner is responsible for training an RNN model with

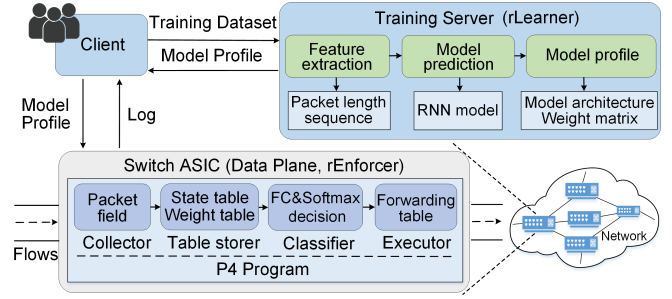


Fig. 3: RIDS architecture and components.

the dataset provided by the client to classify traffic. Different from the traditional deep neural network, the model obtained by rLearner only involves bitwise operations, integer addition/subtraction, and conditional branches. Moreover, we propose novel training designs to enable length-variable decisions (cope with variable time steps). Our model architecture ensures ready deployability on programmable switches.

rEnforcer is the actual component that runs on rLearner-produced models. It utilizes the programmable switches ASICs to analyze traffic and tag packets for each flow. Toward this end, rEnforcer maintains per-flow states in real-time. Specifically, rEnforcer collects the packet length sequence and performs one-hot coding. Then it executes a series of integer addition/subtraction and branch operations to update the state table, which can be considered equivalent to model inference. Finally, rEnforcer makes a forwarding decision for each flow.

Adversary Model and Assumptions. We consider strong adversaries that adopt various attack strategies. During the attack, adversaries may control large-scale botnets so as to use different source addresses over time. Therefore, attackers can change the configuration information such as IPs and application sessions (termed as *dynamic attacks*, case 1 in § VI-B). Furthermore, they might mix different attacks (e.g., Botnet and web attacks) to create new attack vectors. Meanwhile, we do not assume additional collaborations from other Internet entities, such as IP blacklists provided by security vendors.

IV. DESIGN DETAILS OF RIDS

In this section, we elaborate on the detailed design of RIDS.

A. rLearner Design

rLearner is a Recurrent Neural Network (RNN) learning module designed with deployability as a first-class requirement. Thus, rLearner focuses on obtaining a hardware-friendly RNN model whose inference process can be only based on readily available primitives on ASICs, such as bitwise operations or integer additions/subtractions.

1) *Feature Selection and Processing*: To ensure line-speed inference, rLearner should only rely on features that are readily available on the switches. Considering that prior arts have demonstrated that the sequence of packet length is a valid feature as the fingerprint of flow [2], [27], [37], so we choose the packet length sequence as the per-flow feature. Once the input features are determined, a strawman design would be directly performing subsequent calculations on packet length.

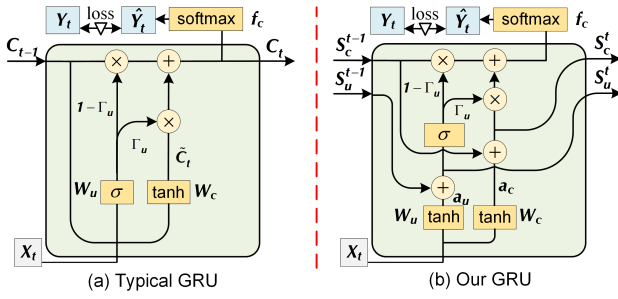


Fig. 4: Traditional GRU architecture and ours.

However, this is neither efficient nor necessary. On the one hand, after the addition operation of multiple packet-length values (e.g., $[0, 1500]$), the range of intermediate results is difficult to predict. As a result, it may take more space to store the calculation process in order to avoid numeric overflow. On the other hand, not every packet-length value is meaningful in terms of model inference. Instead, it matters more which range/interval that a packet-length value belongs to. Therefore, we decide to perform one-hot encoding on the packet length.

Specifically, for a packet p with length l , we perform the following one-hot encoding:

$$\begin{cases} \langle 1, 0, 0, 0 \rangle, & \text{if } l \in [0, 64), \\ \langle 0, 1, 0, 0 \rangle, & \text{if } l \in [64, 128), \\ \langle 0, 0, 1, 0 \rangle, & \text{if } l \in [128, 256), \\ \langle 0, 0, 0, 1 \rangle, & \text{otherwise.} \end{cases} \quad (1)$$

2) *Hardware-Friendly Model Inference*: The second design question is how to realize hardware-friendly RNN model inference. Take Gated Recurrent Unit [6] (GRU, i.e., an RNN model) as an example, Figure 4(a) shows the typical model architecture. In each time step operation, there are four matrix multiplications and two nonlinear activation operations. These operations are difficult to implement on dataplane since programmable switches lack support for floating-point operations and multiplication calculations. In this section, we will elaborate on our hardware-friendly RNN design tailored for programmable switches from two aspects: matrix calculation and activation function.

Matrix Calculation. Matrix multiplication is the main part of model inference. Previous work [19], [24], [29], [34] on network binarization and parameter reduction proposed to use either a binarized matrix (+1 or -1) or ternarized matrix (+1, 0 or -1) to transform multiplication into additions/subtractions. Considering the small number of model parameters, we choose to ternarize the weight matrix to retain the expressive ability of the model. Specifically, the ternarization operation refers to

$$W_i^t = f_t(W_i | \Delta) \begin{cases} +1, & \text{if } W_i > \Delta, \\ 0, & \text{if } |W_i| \leq \Delta, \\ -1, & \text{if } W_i < -\Delta. \end{cases} \quad (2)$$

where Δ is a positive threshold parameter and superscript t means “ternary”. For instance, consider the GRU state vector $V_s = \langle 1, 1, 1, 1 \rangle$, weight $W_s = \langle 1, 1, 0, -1 \rangle$, and $\Delta = 0$, we can get $V_s \times W_s^t = 1 + 1 - 1 = 1$.

Subtraction operation is mainly used to calculate the weight vector when updating the hidden state. Since the weight

vector is activated to $\{0, 1\}$ (explained later in σ function), the subtraction calculation can be an element-wise operation. Specifically, assuming $V_w = \langle 1, 1, 1, 0 \rangle$, we perform the subtraction $1 - V_w = \langle 0, 0, 0, 1 \rangle$.

Activation Function. For the activation function, we use the variant sign function to achieve binarization. We have two types of activation functions for different purposes.

(i) *Weight activation*. This activation function is used to obtain the updated weight matrix, so it binarizes a variable to $\{0, +1\}$ (refer *sigmoid*). Specifically,

$$x_{act} = \sigma(x) = \begin{cases} +1, & \text{if } x \geq \lambda_1, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

(ii) *State activation*. This activation function is used to binarize the hidden state. It contains two states of $\{-1, +1\}$, so the calculation is as follows

$$x_{act} = \tanh(x) = \begin{cases} +1, & \text{if } x \geq \lambda_2, \\ -1, & \text{otherwise.} \end{cases} \quad (4)$$

In (3) and (4), λ_1 and λ_2 are learnable parameters. The corresponding training method will be explained in § IV-A4.

3) *Store Per-Flow State*: To avoid stalling the packet processing pipeline in ASICs, programmable switches have strict restrictions on variable operations that are allowed (aforementioned in § II-B). Specifically, variable operations between *read-write* must be completed in one stage (e.g., only one addition calculation), and a variable can only be accessed once when forwarding an individual packet (constraint of match-action tables). Due to these constraints, the per-flow state cannot be stored as the traditional RNN does. We instead propose a novel technique to fulfill the state maintenance and update.

Considering a traditional GRU where the reset gate is removed, as displayed in Figure 4(a), during the t -th time step, the candidate state can be calculated as follows.

$$\tilde{C}_t = \tanh(W_c \cdot [C_{t-1}, X_t] + b_c) \quad (5)$$

where C_{t-1} is the current hidden state and X_t denotes the t -th input. The weight can be computed as:

$$\Gamma_u = \sigma(W_u \cdot [C_{t-1}, X_t] + b_u) \quad (6)$$

Then, the next hidden state can be calculated as:

$$C_t = \Gamma_u \cdot \tilde{C}_t + (1 - \Gamma_u) \cdot C_{t-1} \quad (7)$$

We notice that there are many calculations between the *read* and *write* of the hidden state C_{t-1} , which is nearly infeasible in programmable switches. For one variable, we can only perform an addition/subtraction between *read-write*. To this end, we decouple the hidden state C_{t-1} and the input X_t . Thus the set matrix W only multiplies with X_t . The results of $W \cdot X_t$ can be used to update the hidden state and weight element-wisely. Specifically, for the t -th time step, we can use (8) to update the state table and weight table using only one addition calculation, as shown in Figure 4(b).

$$\begin{cases} a_c = \tanh(W_c \cdot X_t + b_c) \\ a_u = \tanh(W_u \cdot X_t + b_u) \end{cases}, \begin{cases} S_c^t = S_c^{t-1} + a_c \\ S_u^t = S_u^{t-1} + a_u \end{cases} \quad (8)$$

With the updated content written to the memory, subsequent calculations do not require any *read-write* operations in this time step. By repeating such a process, we activate the weight matrix to $\{0, 1\}$ as (9).

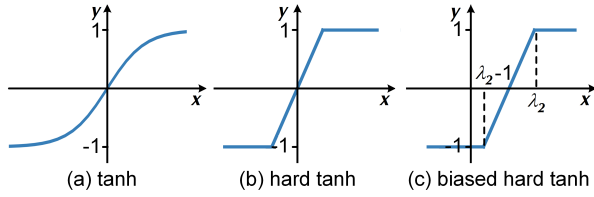


Fig. 5: The biased hard tanh activation function.

$$\Gamma_u = \sigma(S_u^t) \quad (9)$$

Then the output can be calculated as shown in (10).

$$y = \Gamma_u \cdot S_c^t + (1 - \Gamma_u) \cdot S_c^{t-1} \quad (10)$$

Eventually, after fully connected layer f_c and softmax, we can obtain the final classification results.

4) *Backward Propagation and Parameter Update*: We detail how to perform backward propagation and parameter updates. **Activation Function Parameter.** To meet the constraints that the switch only supports integer addition, we can use hard tanh instead of tanh in (4), as Figure 5 shows, which can be reshaped and shifted with an offset of $\lambda_2 - 0.5$ to get the biased hard tanh in Figure 5(c). Now the gradient of λ_2 in tanh can be derived by the chain rule as listed in (11)

$$\frac{\partial L}{\partial \lambda_2} = \frac{\partial L}{\partial A} \frac{\partial A}{\partial \lambda_2} \quad (11)$$

where L represents the loss function and $\frac{\partial L}{\partial A}$ denotes the gradients from hard tanh. Then we can compute the derivative $\frac{\partial A}{\partial \lambda_2} = -1$. So update λ_2 reference $\lambda_2 = \lambda_2 + \eta_2 \frac{\partial L}{\partial \lambda_2}$, where η_2 is the learning rate.

For σ activation function in (3), the derivative can be calculated using a straight-through estimator. Specifically, during the backward propagation, we can directly calculate the gradient of the original sigmoid function ($S(x) = \frac{1}{1+e^{-x}}$) and use this derivative to update parameter λ_1 .

Weight Matrix. For ternary weight, we update the matrices and parameter Δ in (2) respectively. (i) We use the straight-through estimator to get the gradient of the full-precision matrix and update W . During forward propagation, new ternary weights W_{new}^t can be obtained based on the updated full-precision matrix W_{new} . To achieve this process, we will store both W and W^t during model training.

(ii) To update the parameter Δ , we can use the clip function (as shown in Figure 6(b)) to approximate the middle part of the f_t in (2) where $x \in [-\Delta, \Delta]$. The gradient of Δ can be derived by (12)

$$\frac{\partial L}{\partial \Delta} = \frac{\partial L}{\partial A} \frac{\partial A}{\partial W^t} \frac{\partial W^t}{\partial \Delta} \quad (12)$$

where L represents the loss function and $\frac{\partial L}{\partial A}$ denotes the gradients from hard tanh, as well as $\frac{\partial A}{\partial W^t}$ represents the derivative of A to W^t . When $x \in [-\Delta, \Delta]$, given $f_t^{clip}(W) = \frac{W}{\Delta}$, $\frac{\partial W^t}{\partial \Delta} = -\frac{W}{\Delta^2}$. Then, we can update the parameter as

$$\Delta' = \Delta + \eta_w \frac{\partial L}{\partial A} \frac{\partial A}{\partial W^t} \frac{W}{\Delta^2} \quad (13)$$

where η_w is the learning rate.

Training Loss. Assume that the model output $\hat{Y}_t$ for the input X_t of each time step $t \in [1, n]$, the predicted result $\hat{Y}_{pre}$ of

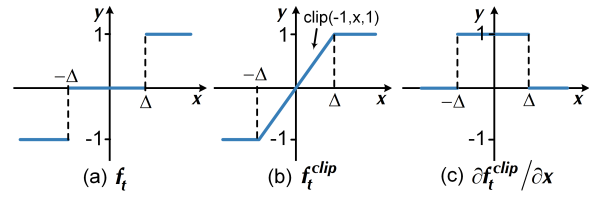


Fig. 6: Approximate ternary weight to clip function.

flow X is 1 (if $\exists t, \hat{Y}_t = 1$), otherwise 0. Also, we denote the ground-truth label as Y_{gt} for flow X . The model loss is designed as $MSE(\hat{Y}_{pre}, Y_{gt}) + \sum_{t \in [1, n]} f(t) \cdot (1 - \hat{Y}_t) \cdot Y_{gt}$, so that samples with high confidence can be identified in small time steps, where $f(\cdot)$ is a decreasing function, e.g., $f(\cdot) = \frac{1}{t}$.

B. rEnforcer Design

rEnforcer is the dataplane design of RIDS. It is responsible for shaping the traffic based on the model inference results. We depict the architecture of rEnforcer in Figure 7. rEnforcer has two pipelines. The first pipeline, termed *S-pipeline*, is responsible for storing and updating the per-flow state. The second pipeline, termed *D-pipeline*, determines the forwarding action based on the inference result. Specifically, S-pipeline contains three tables: (i) the state table that maintains the hidden state for each flow; (ii) the weight table that holds the weight to calculate subsequent outputs; (iii) the forwarding table stores the per-flow forwarding action (such as “nop” and “drop”) and it is initially empty. If a 5-tuple session index is matched by the forwarding table, the corresponding packet is tagged with the action and forwarded directly to the output port of the D-pipeline. Otherwise, the packet traverses the feature extraction modules of the S-pipeline before forwarding. Packets entering the D-pipeline perform prediction and update the forwarding table accordingly. Next, we discuss each module in detail.

1) *Feature Collector*: As in Eq.(1), we perform one-hot encoding for each incoming packet. Towards this end, our compiler will construct a match-action table with 4 entries. For each entry, we use a *range match* to store the range of values in SRAM (Static Random-Access Memory) and compare the per-packet length field (two bytes). The matches associated with the entries will depend on the embedding vector defined in the one-hot encoding. Finally, one packet will be matched with one specific vector since these ranges do not overlap.

2) *Matrix Processor*: The rEnforcer scheduler maintains three matrices, W_c , W_u , and f_c with ternary weight (i.e., $\{-1, 0, 1\}$) that can be stored on the switch ASICs. We use match-action to implement the calculation process and set the *default rule* to match all packets. 1 and -1 correspond to the addition and subtraction primitives, respectively, and 0 means no operation. To reduce the number of used stages (switch resources), we aggregate calculations without mutual dependencies. For instance, to calculate $1 + 2 + 3 + 4$, we can first calculate $1 + 2$ and $3 + 4$ in the first stage, then calculate $3 + 7$ in the second stage. This design allows us to complete δ calculations in $\log \delta$ stages. This can also be applied to two-dimensional matrix computation. For example, to calculate

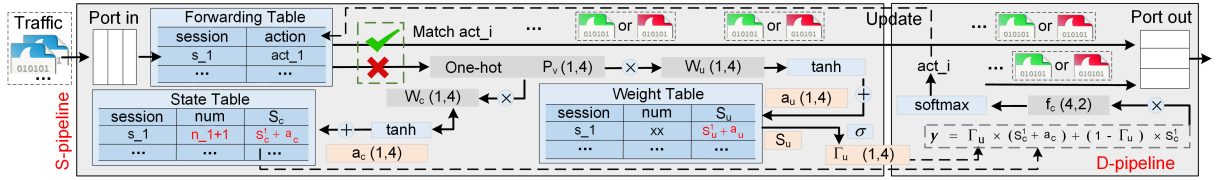


Fig. 7: The architecture of the rEnforcer scheduler designed to deploy RNN models in dataplane.

$X \times W_t$, where X and ternary matrix W_t are organized as (1,4) and (4,4), respectively, we only need two stages. This is because the columns of W_t are independent of each other and therefore can be calculated in parallel.

3) *Table Design*: There are three tables maintained in rEnforcer, state table, weight table, and forwarding table. Among them, the state and weight tables have a similar structure. The first two columns in both tables are the 5-tuple index and the counter that records the number of packets for each flow indexed by its 5-tuple. The third column of the state table and weight table are hidden state and update weight, respectively. As for the forwarding table, each entry contains a 5-tuple index and action. The action essentially is metadata that belongs to the maintenance information of the packet. The compiler in RIDS will generate a read/write register for each entry in three tables and initialize them as empty, as well as be reset when flow timeout.

For the index of all three tables, we cannot directly use the 5-tuple key since it contains a lot of bytes (e.g., 13-byte in IPv4). Therefore, we use the CRC32 algorithm to compute the hash value of the 5-tuple as an index. Due to the constraint that the index of a single array cannot exceed 65,536 (2^{16}) and the possible hash collision of CRC32, we use a second hash (CRC16) to prevent the hash collision. Specifically, when a packet arrives, if the flow count corresponding to the first hash is zero (i.e., it is a new entry), we directly update the corresponding state. Otherwise, we compute CRC16 of the packet and compare it with the pre-stored second hash value. If both hashes match, we update the state accordingly. We count collision times in § VI-C, and results show that using an additional hash can indeed effectively reduce collisions.

4) *Conditional Branch*: To fulfill the activation function in (3) and (4), we perform an element-wise operation (if-else branch) for the vector. For each element, we use *control flow* logic to match the range (λ_1 or λ_2) and use *modified action* to update the activated value. Specifically, we modify variables to $\{0, 1\}$ in the sigmoid function of (3). And for the tanh function of (4), we activate variables to $\{-1, 1\}$.

5) *Softmax*: In the D-pipeline, we perform model inference for each flow and determine its action. In model inference, the softmax operation is executed after the fully connected layer f_c . However, we need to represent negative numbers properly in order to support number comparisons, since the switch ASICs do not have native support for negative numbers. Thanks to the one-hot encoding detailed in § IV-A1, we can be aware of the range of the intermediate calculation results. For instance, suppose RIDS performs the state calculation for the first dozens of packets of one flow, the intermediate and

final results will not exceed 128. Thus, we can afford to use the highest bit of a byte to represent the sign: 1 stands for negative numbers and 0 for positive values.

V. IMPLEMENTATION

We fully implemented a prototype of RIDS in about ~2200 lines of P4 codes, ~3100 lines of Python codes, and ~2500 lines of C++ codes. Our code is available online¹. Figure 8 depicts the P4 pseudo-code of key compiler modules for RIDS. For instance, three code snippets in Figure 8(a): (i) the one-hot header as the feature vector; (ii) two different hash algorithms for index and key verification; (iii) *tanh* activation function.

The main function shown in Figure 8(e) describes the overall packet processing logic. At the beginning, RIDS counts the packets, collects per-flow features, and updates the corresponding states. Then RIDS will perform model prediction and assign specific forwarding action for the subsequent packets of the flow. We merge some modules that can run parallelly to reduce the utilization of switch hardware resources. Next, we elaborate on these optimizations in detail.

Simplify Value Comparison. In § IV-B5, we propose using the complement store negative numbers to fulfill value comparison. A vanilla design may directly branch calculation according to each situation (such as $\{+, +\}$, $\{+, -\}$, $\{-, -\}$). However, this is inefficient and unnecessary. Therefore, we simplify the comparison process. Specifically, to compare two variables (8 bits) C_x and C_y , (i) calculate $C_x = C_x - C_y$, (ii) then compare C_x with 128 (0x80) to get the final result. The corresponding pseudo-code is shown in Figure 9(a).

Record Forwarding Actions in the State Table. To reduce the utilization of memory access, we consider storing forwarding actions directly in the state table. Thus, we store the packet count in the state table. Then, we set “action” as an additional field in the state table to define the specific forwarding action. Thus we do not need to maintain a separate forwarding table and reduce the repeated storage of the index hash.

Merge Tanh and State Updates. Updating the per-flow state involves (i) using tanh to activate the matrix calculation result mc (or mu) in Figure 9(c); and (ii) adding the activation value to the state, i.e., (8). We combine these two processes to reduce the usage of hardware stages in the switch pipeline. Specifically, we first compare mc (or mu) with activation threshold λ_2 , then directly perform element-wise +1 or -1 for the state vector, as shown in Figure 9(c).

Simplify Decision Making. In addition, we also optimize the decision-making process. Before the fully connected layer in our RNN model, the output y needs to be calculated according

¹See repository <https://github.com/Secbrain/RIDS/>

```

header onehot{
  bit<2> v1;
  bit<2> v2;
  bit<2> v3;
  bit<2> v4;
} //<v1,v2,v3,v4>
control hash
(5_tuple, index, key) {
  index=crc32(5_tuple);
  key=crc16(5_tuple);
} //hash

register<8>  $\lambda_2$ ;
control tanh(x, x_a) {
  if x >=  $\lambda_2$ 
    x_a = 1;
  else x_a = -1;
} //tanh
(a) One-hot and hash

//packet length to one-hot
table pktlen_onehot{
  key = {
    ipv4.total_len : range;
  }
  actions = { //four vectors
    //choose one vector
  }
} //do_flow_cnt
register<8> (65536) cnt_tab;
metadata<16> p_ind, p_key;
metadata<16> 5_tuple;
control count{
  hash(5_tuple, p_ind, p_key);
  cnt_tab[p_ind]++;
}
(b) Feature and count

//do_calc_mat
control do_calc_mat
(m1, m2, m3, m4) {
  m1+=m2;
  m3+=m4;
  m1+=m3;
} //pairwise parallel
//update_state
register<8> sta, wei;
register<8> sta_ori;
metadata<8> oc, ou, ac, au;
control do_update_state{
  tanh(oc, ac);
  tanh(ou, au);
  sta_ori=sta;
  sta+=ac;
  wei+=au;
}
(c) Matrix and state

header forward{
  bit<4> action;
} //Forwarding

register<8> sta, wei;
register<8> sta_ori;
metadata<4> y, r_dec,  $\Gamma_u$ ;
control do_decision{
  sigmoid(wei,  $\Gamma_u$ );
  //element-wise
  if  $\Gamma_u=1$ 
    y=sta;
  else
    y=sta_ori;
  do_calc_mat(y);
  softmax(y, r_dec);
  flow.forward = r_dec;
} //do_decision
(d) Decision

register<8>  $\lambda_1$ ;
control sigmoid(wei,  $\Gamma_u$ )
{
  if wei >=  $\lambda_1$ 
     $\Gamma_u = 1$ ;
  else  $\Gamma_u = 0$ ;
} //sigmoid

control ingress
{
  count();
  do_calc_mat();
  do_update_state();
  do_decision();
} //main
(e) Activation and main

```

Fig. 8: The RIDS compiler processes the key language constructs and generates P4 implementations.

```

control compare(a, b, res)
{
  a=b;
  if a < 0x80
    res=1;
  else {res=0}
} //res=1 means ">"
(a) Simplify comparison

register<8>  $\lambda_2$ ;
metadata<8> mc, mu;
metadata<1> resc, resu;
register<8> sta, wei;
control{
  compare(mc,  $\lambda_2$ , resc)
  if resc==1
    sta++;
  else {sta--;}
  compare(mu,  $\lambda_2$ , resu)
  if resu==1
    wei++;
  else {wei--;}
} //update sta and wei
//table in parallel
updates
(c) Merge tanh and state

```

Fig. 9: P4 code snippets of some optimized modules.

TABLE I: Datasets used in our evaluation.

IDS 17&18	Benign	Human interaction traffic produced by B-Profile UDP, TCP, HTTP
	DDoS	Hulk, GoldenEye, Slowloris, Slowhttptest, Heartbleed
	DoS	Remote shell, File, Key logging
	Botnet Patator Web	FTP, SSH Brute-force, XSS, SQL inject
USTC	Benign	BitTorrent, Facetime, FTP, Gmail, MySQL, Outlook, Skype, SMB, Weibo, World of Warcraft
	Malware	Cridex, Geodo, Htbot, Miuref, Neris, Nsis-ay, Shifu, Tinba, Virut, Zeus

to (10). Due to sigmoid activation of (3), each value in vector Γ_u is either 0 or 1. Therefore, this matrix calculation can be transformed into an element-wise assignment. For each position in y , if Γ_u is 1, assign S_c^{new} ; otherwise, assign S_c^{ori} . Figure 9(b) shows the specific implementation.

After implementation optimization, some modules share a common stage. For instance, the state storer and decision module have many shared operations, so they are combined together, detailed explanations in § VI-C.

VI. EVALUATION

In this section, we extensively evaluate RIDS including comparing state-of-the-art (SOTA) models before deploying switches in § VI-A, real-time detection on switches (in terms of DDoS defense, intrusion detection, and malware identification scenarios) in § VI-B, overhead evaluation (hardware utilization, throughput, latency, *etc.*) in § VI-B.

Testbed Setup. Our physical testbed contains (i) one host that serves as the packet generator with three 100 Gbps Mellanox ConnectX-5 network cards, two NICs for sending attack traffic, and one NIC for benign traffic. (ii) A receiving host with three 100 Gbps Mellanox ConnectX-5 network cards, only one is enabled at DDoS evaluation. (iii) A Tofino switch (Wedge 100BF-32X), deploying RIDS, has 32 100 Gbps ports. The above hosts are running Ubuntu 20.04.1 LTS with Linux kernel 5.4.0. They all have Intel Xeon(R) E5-2620V3 CPU (2.10 GHz, 8 cores), and 64 GB of memory. We use MoonGen [10] to replay PCAP for traffic generation.

Datasets. The datasets used for evaluation are in Table I, including two intrusion detection datasets and a malware identification dataset. (i) IDS2017 [13] and IDS2018 [12] include five types of intrusion attacks, and a series of legitimate traffic of human interactions produced by B-Profile system [13]. (ii) USTC-TFC dataset [41] contains network traffic generated

by malware and benign services from real-world connections. Specifically, it involves ten types of malware traffic (*e.g.*, Cridex, Zeus, *etc.*) and ten categories of benign applications (*e.g.*, Facetime, Skype, Weibo). If not otherwise stated, the dataset division ratio refers to *train* : *test* = 6:4.

Evaluation Metrics. Our primary evaluation metric for RNN model performance is the overall Accuracy *Acc*, precision *Pre*, recall *Rec*, and F1-score *F1*. On the dataplane, we consider additional evaluation metrics including the hardware resource usage, latency, throughput, and so on.

Settings. The hyperparameters for RIDS and the typical GRU, the settings refer to that the maximum time step is 128, the dimensions of input and hidden layer are both 4, the learning rate is 0.001, the batch size is 64, and the number of epochs is 100. The experiments in § VI-A are performed 5 times with different random seeds, both the performance mean and standard variance are reported.

A. Compare with SOTA

We first compare the detection effect of RIDS with state-of-the-art (SOTA) models. As stated in § II, Mousika [44] proposes to deploy the Binary Decision Tree (distilled from GRU is used) on switches for packet classification. Kitsune [31] dedicates to developing a low-complexity DNN model to reduce overhead and proposes to use AutoEncoder to detect abnormal behavior. HyperVision [15] designs the flow interaction graph to detect malicious traffic. Mousika, HyperVision, and Kitsune are representative baselines with aspects of detection method and high bandwidth adaptation, as shown in Figure 1. We also compare the typical GRU [6] to explore the accuracy loss of our design. The results are summarized in Table II. Compared to a typical GRU, RIDS exhibits some accuracy drop due to the integer-weights design. The recall is from 98.56% to 98.31% in the IDS dataset, yet the slight precision drop ($\sim 0.02\%$) means RIDS does not introduce mass false positive.

TABLE II: Compare with baseline models.

	Methods	Accuracy	Precision	Recall	F1-score
IDS 17&18	GRU (offline)	99.25±0.16	99.89±0.14	98.56±0.23	99.22±0.18
	HyperVision	97.05±0.46	96.36±0.34	97.82±0.28	97.11±0.36
	Mousika	92.75±0.43	91.37±0.33	92.64±0.49	91.96±0.45
	Kitsune	97.25±0.31	98.46±0.25	96.02±0.36	97.22±0.34
	RIDS	99.03±0.22	99.87±0.19	98.31±0.25	99.09±0.23
USTC	GRU (offline)	99.43±0.11	99.55±0.08	99.22±0.17	99.31±0.15
	HyperVision	98.85±0.21	98.69±0.25	98.90±0.15	98.85±0.19
	Mousika	94.25±0.52	93.53±0.50	95.12±0.36	94.30±0.42
	Kitsune	98.38±0.28	98.71±0.20	98.01±0.40	98.33±0.31
	RIDS	99.15±0.17	99.50±0.10	98.82±0.23	99.16±0.15

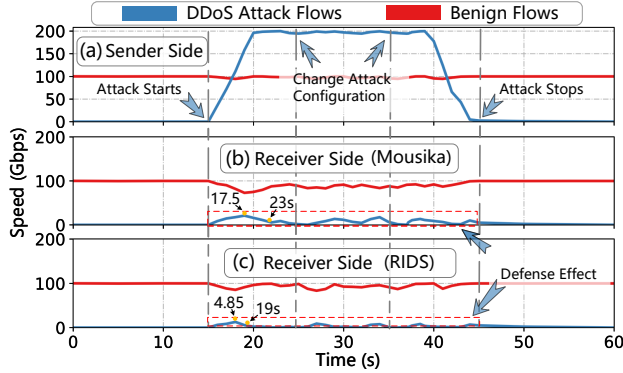


Fig. 10: DDoS mitigation evaluation.

There is little difference in detection performance of HyperVision and Kitsune, while they only support limited bandwidth (as illustrated in Figure 1). For the deployable models (into switches), RIDS significantly outperforms Mousika in these detection metrics, *e.g.*, the IDS *Acc* of RIDS is $\sim 6.2\%$ higher than Mousika. This is because RIDS considers the stateful RNN that could capture the contextual semantics of traffic, which is lacking in stateless Mousika. Overall, RIDS realizes a good trade-off between accuracy and efficiency (*i.e.*, hardware deployability) compared to HyperVision, Kitsune, and Mousika.

B. Evaluation on Different Scenarios

To further demonstrate its effectiveness, we evaluate RIDS in three case studies, addressing complicated attack scenarios. Given typical GRU, Kitsune, and HyperVision do not support 100 Gbps per port, subsequent experiments mainly compare with Mousika (due to its deployability on switches).

Case 1: DDoS Defense. To develop DDoS defense evaluations, we launch the diverse DoS and DDoS attacks mentioned in Table I to saturate the bottleneck of bandwidth. The attacker uses three 100 Gbps NICs to generate traffic, two for DDoS and one for benign, with a peak throughput of 300 Gbps. Only one 100 Gbps network card is enabled for the victim to receive packets. Particularly, strong adversaries may control large-scale botnets to dynamically use different source addresses and sessions over time. We take this into consideration and conduct the simulation with roughly 50K different IP addresses.

Figure 10 describes how the throughputs (denoted as S in Gbps) of benign and DDoS traffic change corresponding to the time duration (denoted as t in seconds). The benign flow is marked in red and maintained as 100 Gbps approximately. The DDoS traffic is marked in blue and initialized when $t = 15$. Figure 10(a) shows the behavior on the sender side. The

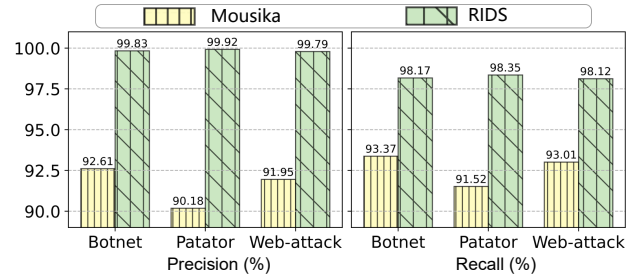


Fig. 11: Precision and Recall in intrusion attacks evaluation.

network utilization of attack traffic reaches around 200 Gbps bandwidth when $t = 20$. Note that the attacker changes the configuration (*e.g.*, using new source IPs and sessions) when $t = 25$ and $t = 35$. The volume of DDoS attacks starts to reduce when $t = 40$, and eventually the attacks end at around $t = 45$. Figure 10(b) and (c) show how Mousika and RIDS prevent DDoS attacks at the receiver side. We find that Mousika exhibits $\{S_{max}^{DDoS} = 17.5, S_{t=23}^{DDoS} = 0\}$. However, the mitigation effect of RIDS is more effective, manifested as $\{S_{max}^{DDoS} = 4.85, S_{t=19}^{DDoS} = 0\}$, which can detect the attack flow earlier and reduce the peak value of DDoS traffic.

Case 2: Intrusion Detection. We evaluate the detection effect of RIDS and Mousika when suffering from intrusion attacks. Figure 11 displays per-type results where the yellow mark refers to Mousika and the green mark represents RIDS. It is clear that RIDS fulfills the better performance for both precision and recall. Particularly, Mousika usually obtains $Rec > Pre$ yet RIDS tends to $Pre > Rec$. Nonetheless, two metrics of RIDS are about 5%~9% higher than Mousika.

Case 3: Malware Identification. Figure 12 shows the Receiver Operating Characteristic (ROC) curve and F1-score for malware detection. We observe that the F1 of RIDS is consistently better than Mousika over time. The ROC curve shows that RIDS can achieve high true positives while fewer false positives.

These case studies demonstrate that flow-state-aware RIDS achieves better performance compared to stateless Mousika. Another main reason is that the tightly-coupled module design of RIDS does not cause much classification boundary loss, thus retaining the most performance of full-precision GRU (visualization results in aforementioned anonymous URL).

C. Overhead Evaluations

The overhead results of RIDS system are collected from the real hardware switch using the P4 defense program. The evaluation below demonstrates that RIDS is practical on today's hardware switches as it introduces low overhead.

Hardware Utilization. For the hardware resource utilization, all of these hardware resources are spread across 11 hardware stages in the switching pipeline. Specifically, the matrix processor utilizes two switch stages since it contains a series of calculations for model inference. And the modules of state tables and decision-making need more SRAM (83) and hash units (297) to maintain the per-flow state. Since the softmax module mainly relies on exact matching and gateways, it does not consume any of Ternary Content Addressable Memory (TCAM) resources on the switch. Overall, RIDS uses 22.92%

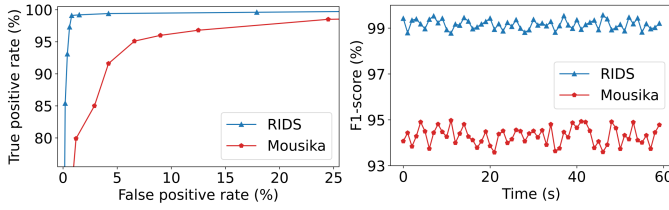


Fig. 12: ROC and F1 results for malware detection.

Arithmetic Logical Units (ALUs) for header computation, 4.86% TCAM, and 11.11% of the CRC hash units, as well as 9.38% SRAM (Table III). Therefore, RIDS consumes a small number of hardware resources, which means RIDS is practical in current modern switches and is promising to extensibility.

Hash Index Collisions. Considering there will be hash collisions in the table index, we record the original 5-tuple with the corresponding active timestamp to counter hash collision numbers. Based on the experiments in § VI-B which roughly involve $\sim 50K$ IP addresses, it occurs 65 and 374 times of hash collision when only using CRC32 or CRC16, respectively. While the two algorithms are used together, hash collisions are reduced to 2 times. Introducing multiple algorithms to alleviate hash collision is horizontally scalable, and we are not against collaborations with the stateless scheme (*e.g.*, Mousika) when a hash collision occurs in the future.

Throughput and Latency. We enable three 100 Gbps NICs both at the sender and receiver sides to build a 300 Gbps link, and replay the five-day traffic of IDS2017 to test the throughput and latency. We also configure different model parameters (rEnforcer) into the switches and plot the throughput diagram in the left part of Figure 13. Regardless of the model parameter profile, RIDS realizes the real-time traffic detection and can saturate the data plane, *i.e.*, 300 Gbps. The pipelined nature of the switch hardware ensures RIDS achieves full bandwidth, that is, RIDS realizes line-speed DL model inference.

We also measure the end-to-end latency in the right part of Figure 13, and the results show that RIDS program only introduces about 96 ns in total. Among them, the state tables and decision-making component consume the most latency overhead (50 ns) as they use more stages. From the switch hardware perspective, a *stage* generally needs about 9 ns processing latency. Both *control flow* and *register* only introduce less than 5 ns latency. The other four components including Feature, Softmax, Count, and Matrix impose less than 20 ns. Overall, RIDS introduces the imperceptible extra latency, especially considering network RTTs (round-trip times) are typically at the order of milliseconds in the Internet core.

VII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

Security of RIDS. The intrusion detection of RIDS is constructed on the *shared* dataplane of the programmable switch. First, the adversary may insert packets to disturb the collected flow sequence [17], [42]. Meanwhile, network-induced phenomena are also very common in high-speed bandwidth, such as packet loss and retransmission. The problematic traffic may cause model misclassifications. Therefore,

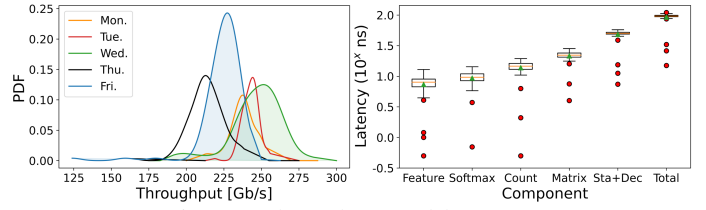


Fig. 13: Throughput and latency.

TABLE III: Hardware resource utilization breakdown in RIDS.

Modules	Switch Resource Usage Breakdown						
	Stage	xBar	Hash	Gateways	VLIW	SRAM	TCAM
Feature Collector	1	4	0	0	1	1	3
State Table+Decision	6	75	297	9	29	83	8
Conditional Branch	1	7	4	2	4	1	3
Matrix Processor	2	4	0	6	8	1	3
Softmax	1	1	8	3	3	0	0
Total	11	91	4.97%	10.94%	8.07%	9.38%	4.86%

some robust training methods [52] can be combined in RIDS to improve fault tolerance. Second, we discuss the security of the forwarding table. As mentioned in § IV-B, when the packet arrives, it will traverse the S-pipeline and D-pipeline in rEnforcer and update its flow state and forwarding action. To ensure the freshness of the forwarding decision, we reset the forwarding table periodically for long-term inactive flows, or when the number of packets in a flow exceeds a certain value.

Extensibility of RIDS. Based on our basic modules that are already decoupled (in § IV-A), there are some possible directions for expansion. (i) Feature extraction. On the one hand, more features can be extracted from network flows, *e.g.*, TCP Flag and time to live. On the other hand, other encoding methods can also be utilized, instead of the one-hot that only focuses on the range information. (ii) Model architecture. RIDS is not limited by any specific model architecture. For example, emerging techniques such as transformer [26] show more powerful sequence modeling capabilities, and we leave it to future work to deploy more model architectures in the switches.

Practicality of RIDS. Model training of RIDS is offline, and trained model parameters will be re-delivered from the control plane to the data plane when need to be updated. Even the rule-based IDS needs to re-deliver the updated rule configuration from the control plane to the data plane [28], [49]. RIDS (the offline training and online deployment) does not violate current design trends. In practice, additional switches can be used to perform flow state replication/backup to ensure that legitimate flows are not interrupted when updating the model [49].

VIII. CONCLUSION

In this paper, we propose RIDS, a novel intrusion detection system that leverages the emerging hardware primitives to directly perform RNN model inference on the programmable switches. By employing a model and dataplane primitive co-designed approach, RIDS realizes the deep-learning-powered traffic classification capability inside the network with negligible overhead. We implement a prototype of RIDS and evaluate it extensively on our testbed. The results show that RIDS can achieve $>99\%$ F1 score for intrusion/malware detection, while only imposing nanoseconds of latency on the physical testbed with 300 Gbps maximum throughput.

REFERENCES

- [1] Aristide Tanyi-Jong Akem, Michele Gucciardo, and Marco Fiore. Flowrest: Practical flow-level inference in programmable switches with random forests. In *INFOCOM*, pages 1–10. IEEE, 2023.
- [2] Diogo Barradas, Nuno Santos, Luís Rodrigues, et al. Flowlens: Enabling efficient flow classification for ml-based network security applications. In *NDSS*. The Internet Society, 2021.
- [3] Pat Bosshart, Dan Daly, et al. P4: programming protocol-independent packet processors. *Comput. Commun. Rev.*, 44(3):87–95, 2014.
- [4] Coralie Busse-Grawitz, Roland Meier, Alexander Dietmüller, Tobias Bühler, and Laurent Vanbever. pforest: In-network inference with random forests. *CoRR*, abs/1909.05680, 2019.
- [5] Jiale Chen et al. Latency-driven optimization of switching pipeline design in network chips. In *RTSS*, pages 344–355. IEEE, 2022.
- [6] Kyunghyun Cho, Bart van Merriënboer, et al. On the properties of neural machine translation: Encoder-decoder approaches. In *SSST@EMNLP*, pages 103–111. Association for Computational Linguistics, 2014.
- [7] Bruno Loureiro Coelho and Alberto Schaeffer-Filho. BACKORDERS: using random forests to detect ddos attacks in programmable data planes. In *EuroP4@CoNEXT*, pages 1–7. ACM, 2022.
- [8] The P4 Language Consortium. P4 bmv2 behavioral model. [EB/OL], 2020. <https://github.com/p4lang/behavioral-model>.
- [9] Aditya Dhakal and K. K. Ramakrishnan. Netml: An NFV platform with efficient support for machine learning applications. In *NetSoft*, pages 396–404. IEEE, 2019.
- [10] Paul Emmerich, Sebastian Gallenmüller, Daniel Raumer, et al. MoonGen: A Scriptable High-Speed Packet Generator. In *Internet Measurement Conference 2015 (IMC'15)*, Tokyo, Japan, October 2015.
- [11] Seyed Kaveh Fayaz, Yoshiaki Tobioka, et al. Bohatei: Flexible and elastic ddos defense. In *USENIX Security Symposium*, pages 817–832. USENIX Association, 2015.
- [12] Canadian Institute for Cybersecurity. Cse-cic-ids2018 on aws. [EB/OL], 2018. <https://www.unb.ca/cic/datasets/ids-2018.html>.
- [13] Canadian Institute for Cybersecurity. Intrusion detection evaluation dataset (cicids2017). [EB/OL], 2018. <https://www.unb.ca/cic/datasets/ids-2017.html>.
- [14] Chuanpu Fu, Qi Li, et al. Realtime robust malicious traffic detection via frequency domain analysis. In *CCS*, pages 3431–3446. ACM, 2021.
- [15] Chuanpu Fu, Qi Li, and Ke Xu. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. In *NDSS*. The Internet Society, 2023.
- [16] Arpit Gupta, Rob Harrison, et al. Sonata: query-driven streaming network telemetry. In *SIGCOMM*, pages 357–371. ACM, 2018.
- [17] Mark Handley, Vern Paxson, and Christian Kreibich. Network intrusion detection: Evasion, traffic normalization, and end-to-end protocol semantics. In *USENIX Security Symposium*.
- [18] Kuo-Feng Hsu et al. Contra: A programmable system for performance-aware routing. In *NSDI*, pages 701–721. USENIX Association, 2020.
- [19] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, et al. Binarized neural networks. In *NIPS*, pages 4107–4115, 2016.
- [20] Xin Jin, Xiaozhou Li, et al. Netchain: Balancing key-value stores with fast in-network caching. In *SOSP*, pages 121–136. ACM, 2017.
- [21] Xin Jin, Xiaozhou Li, Haoyu Zhang, et al. Netchain: Scale-free sub-rtt coordination. In *NSDI*, pages 35–49. USENIX Association, 2018.
- [22] Qiao Kang et al. Programmable in-network security for context-aware BYOD policies. In *USENIX Security Symposium*, pages 595–612. USENIX Association, 2020.
- [23] Jong-Hyoun Lee and Kamal Singh. Switchtree: in-network computing and traffic analyses with random forests. *Neural Computing and Applications*, pages 1–12, 2020.
- [24] Fengfu Li et al. Ternary weight networks. *CoRR*, abs/1605.04711, 2016.
- [25] Hongda Li, Hongxin Hu, Guofei Gu, et al. vnids: Towards elastic security with safe and efficient virtualization of network intrusion detection systems. In *CCS*, pages 17–34. ACM, 2018.
- [26] Xinjie Lin, Gang Xiong, et al. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *WWW*, pages 633–642. ACM, 2022.
- [27] Chang Liu et al. Fs-net: A flow sequence network for encrypted traffic classification. In *INFOCOM*, pages 1171–1179. IEEE, 2019.
- [28] Zaoxing Liu, Hun Namkung, et al. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches. In *USENIX Security Symposium*, pages 3829–3846. USENIX Association, 2021.
- [29] Zechun Liu, Zhiqiang Shen, et al. Reactnet: Towards precise binary neural network with generalized activation functions. In *ECCV (14)*, volume 12359, pages 143–159. Springer, 2020.
- [30] Rui Miao, Hongyi Zeng, Changhoon Kim, Jeongkeun Lee, and Minlan Yu. Silkroad: Making stateful layer-4 load balancing fast and cheap using switching asics. In *SIGCOMM*, pages 15–28. ACM, 2017.
- [31] Yisroel Mirsky et al. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *NDSS*. The Internet Society, 2018.
- [32] Srinivas Narayana et al. Language-directed hardware design for network performance monitoring. In *SIGCOMM*, pages 85–98. ACM, 2017.
- [33] D. Project. Dpdk: Data plane development kit. [EB/OL], 2010. <http://dpdk.org/> Accessed November 27, 2020.
- [34] Mohammad Rastegari, Vicente Ordonez, et al. Xnor-net: Imagenet classification using binary convolutional neural networks. In *ECCV (4)*, volume 9908, pages 525–542. Springer, 2016.
- [35] Davide Sanvito, Giuseppe Siracusano, and Roberto Bifulco. Can the network be the AI accelerator? In *NetCompute@SIGCOMM*, pages 20–25. ACM, 2018.
- [36] Tal Shapira and Yuval Shavitt. Flowpic: Encrypted internet traffic classification is as easy as image recognition. In *INFOCOM Workshops*, pages 680–687. IEEE, 2019.
- [37] Meng Shen, Mingwei Wei, Liehuang Zhu, et al. Classification of encrypted traffic with second-order markov chains and application attribute bigrams. *IEEE TIFS*, pages 1830–1843, 2017.
- [38] Giuseppe Siracusano and Roberto Bifulco. In-network neural networks. *CoRR*, abs/1801.05731, 2018.
- [39] John Sonchack et al. Scaling hardware accelerated network monitoring to concurrent and dynamic queries with *flow. In *USENIX Annual Technical Conference*, pages 823–835. USENIX Association, 2018.
- [40] Zhuoxue Song, Ziming Zhao, et al. I2RNN: An Incremental and Interpretable Recurrent Neural Network for Encrypted Traffic Classification. *IEEE Transactions on Dependable and Secure Computing*, 2023.
- [41] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng. Malware traffic classification using convolutional neural network for representation learning. In *ICoIN*, pages 712–717. IEEE, 2017.
- [42] Zhongjie Wang, Shitong Zhu, Yue Cao, et al. Symtcp: Eluding stateful deep packet inspection with automated discrepancy discovery. In *NDSS*. The Internet Society, 2020.
- [43] Bruno Missi Xavier, Rafael Silva Guimaraes, Giovanni Comarella, and Magnus Martinello. Programmable switches for in-networking classification. In *INFOCOM*, pages 1–10. IEEE, 2021.
- [44] Guorui Xie, Qing Li, Yutao Dong, et al. Mousika: Enable General In-Network Intelligence in Programmable Switches by Knowledge Distillation. In *INFOCOM*. IEEE, 2022.
- [45] Jiarong Xing et al. Ripple: A programmable, decentralized link-flooding defense against adaptive adversaries. In *USENIX Security Symposium*, pages 3865–3881. USENIX Association, 2021.
- [46] Jiarong Xing, Wenqing Wu, and Ang Chen. Architecting programmable data plane defenses into the network with fastflex. In *HotNets*, pages 161–169. ACM, 2019.
- [47] Zhaoqi Xiong and Noa Zilberman. Do switches dream of machine learning?: Toward in-network classification. In *HotNets*, pages 25–33. ACM, 2019.
- [48] Nofel Yaseen, John Sonchack, and Vincent Liu. Synchronized network snapshots. In *SIGCOMM*, pages 402–416. ACM, 2018.
- [49] Menghao Zhang et al. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *NDSS*. The Internet Society, 2020.
- [50] Ziming Zhao et al. CMD: Co-analyzed IoT Malware Detection and Forensics via Network and Hardware Domains. *IEEE Transactions on Mobile Computing*, 2023.
- [51] Ziming Zhao, Zhaoxuan Li, et al. DDoS Family: A Novel Perspective for Massive Types of DDoS Attacks. *Comput. Secur.*, 2023.
- [52] Ziming Zhao, Zhaoxuan Li, et al. ERNN: Error-Resilient RNN for Encrypted Traffic Detection towards Network-Induced Phenomena. *IEEE Transactions on Dependable and Secure Computing*, 2023.
- [53] Ziming Zhao, Zhuotao Liu, et al. Effective DDoS Mitigation via ML-Driven In-network Traffic Shaping. *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [54] Changgang Zheng and Noa Zilberman. Planter: seeding trees within switches. In *SIGCOMM Posters and Demos*, pages 12–14. ACM, 2021.
- [55] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. An efficient design of intelligent network data plane. In *USENIX Security Symposium*, pages 6203–6220. USENIX Association, 2023.